Functional Versus Imperative Is Solved

Power, Cost, and Shared State in the Paradigm Debate

29 October 2026

1 Two Ways to Sum a List

Here is the same function written twice.

Listing 1: Imperative, in C.

```c
int total(const int *xs, int n) {
  int acc = 0;
  for (int i = 0; i < n; i++)
    acc += xs[i];
  return acc;
}
```

Listing 2: Functional, in Haskell.

```haskell
total :: [Int] -> Int
total []       = 0
total (x : xs) = x + total xs
```

The first is a list of instructions. The second is a pair of equations. Programmers have argued about which is better for most of the history of programming, and the argument is older than most of the languages it is now conducted in. We settle it.

Definition 1.1 (Imperative)**.** A program is *imperative* if its meaning is a sequence of commands, each of which changes a *store*, a map from locations to values. Running it executes the commands in order, and its result is read from the final store.

Definition 1.2 (Functional)**.** A program is *functional* if its meaning is an expression, and the meaning of an expression is its value. It is *pure* if the value of every expression depends only on the values of its subexpressions, so that any subexpression can be replaced by its value without changing the meaning of the whole. Strachey, borrowing from Quine, called this property *referential transparency* [4].

Remark 1.3. The term has since been used in several senses that are not equivalent [3]. We use Strachey's. Both definitions classify programs, not languages. C permits a function that assigns nothing, and Haskell permits one that does nothing but assign. Most languages in use permit both, so the question is not which language to use but which kind of program to write.

2 The Ceasefire of 1937

The first question, whether one style can compute something the other cannot, was settled before either had a machine to run on. Church proposed that the effectively calculable functions are those definable in the λ-calculus [1]. Turing proposed those computable by a machine that reads and writes symbols on a tape, following a table of instructions [5]. The first is a functional language and the second an imperative one.

Theorem 2.1 (The Church–Turing Ceasefire)**.** *A function on the natural numbers is λ-definable if and only if it is computable by a Turing machine.*

Proof. Turing showed that every λ-definable function is computable and every computable function is general recursive [6], and Kleene that every general recursive function is λ-definable [2]. □

Corollary 2.2 (No Exclusive Programs)**.** *Any function computed by a program in one Turing-complete language, functional or imperative, is computed by some program in any other.*

Proof. Each such language computes at least the functions that Turing machines compute, since it is Turing-complete, and at most these, since a Turing machine can run its interpreter. Theorem 2.1 is the case of the λ-calculus and Turing's machines. □

Remark 2.3. The ceasefire was signed in 1937, eleven years before the first stored-program computer ran a program [7]. It concerns which functions can be computed, not how quickly, how clearly, or how often incorrectly. The rest of this paper is about those.

References

[1] A. Church, An unsolvable problem of elementary number theory, *American Journal of Mathematics* 58(2), 1936.

[2] S. C. Kleene, λ-definability and recursiveness, *Duke Mathematical Journal* 2(2), 1936.

[3] H. Søndergaard and P. Sestoft, Referential transparency, definiteness and unfoldability, *Acta Informatica* 27(6), 1990.

[4] C. Strachey, Fundamental concepts in programming languages, *Higher-Order and Symbolic Computation* 13(1–2), 2000. Lecture notes from 1967.

[5] A. M. Turing, On computable numbers, with an application to the Entscheidungsproblem, *Proceedings of the London Mathematical Society* s2-42(1), 1936.

[6] A. M. Turing, Computability and λ-definability, *Journal of Symbolic Logic* 2(4), 1937.

[7] University of Manchester, How a 70-year-old Baby changed the face of modern computing, https://www.manchester.ac.uk/about/news/how-a-70-year-old-baby-changed-the-face-of-modern-computing/, 21 June 2018.